

Nom	Prénom

I. Programmation Dynamique.

Rémy le « Djinakh » aime beaucoup le fromage. Malheureusement, il est très limité dans ses déplacements : il ne peut aller que vers le bas ou vers la droite d’une unité. En supposant qu’il se trouve dans une case de l lignes et c colonnes, on souhaite concevoir un algorithme qui lui permettra de manger le plus possible de fromage. Par exemple, dans la matrice ci-dessous.

						
						
						
						
						

S’il effectue les déplacements ddbbddd (où d signifie “droite” et b signifie “bas”), il récupère 5 fromages, soient ceux aux coordonnées (1, 2), (2, 3), (3, 3), (4, 5) et (5, 5).

Supposons que Rémy commence toujours à la case (1, 1) et termine à la case (l c). Vous pouvez également supposer qu’il n’y a jamais de fromage à la case (1, 1). Pour $i = 1, 2, \dots, l$ et $j = 1, 2, \dots, c$, soit $C(i, j)$ le nombre maximum de fromages que Rémy peut récupérer en se déplaçant de la position (1, 1) à la position (i, j).

- a. Donnez une formule récursive pour calculer $C(i, j)$ (en n'oubliant pas les cas de base).
- b. Donnez un algorithme basé sur la programmation dynamique qui permet de calculer le nombre maximum de fromages que Rémy peut récupérer pour une grille de l lignes et de c colonnes en supposant que la fonction $\text{ExisteFromage}(i, j)$ retourne 1 s'il y a une carotte à la ligne i et la colonne j , ou 0 sinon.
- c. Ecrire un programme avec un langage de votre choix (c, c++, java, python) qui permet à Rémy de se déplacer sur la grille tout en éliminant les fromages qu'il a déjà mangés.
- d. En justifiant brièvement, calculez la complexité de votre algorithme à l'aide de la notation θ